Web Scraping

Prepared with AI Aid.

Agenda

3.1	HTML Structure – Building Blocks of Web Pages
3.2	CSS Selectors – Finding Elements
3.3	BeautifulSoup Methods – Extracting Data
3.4	Real Example – Scraping Book Data
3.5	robots.txt – The Rules of Scraping

What is Web Scraping?

Web scraping is the automated extraction of data from websites by parsing their HTML structure.

Input	Output
▸ A website URL	▸ Structured data (CSV, JSON, DataFrame)
▸ HTML page content	▸ Cleaned, validated records
▸ Target data (prices, titles, links…)	▸ Analysis-ready datasets

Typical Scraping Pipeline:

Send HTTP Request → Receive HTML → Parse with BeautifulSoup → Extract Data → Save / Analyze

💡 **Use scraping only when no API or downloadable dataset is available!**

3.1 HTML Structure – Building Blocks

```
<html>                          ← Root
  <head>                        ← Branch
    <title>Page</title>         ← Leaf
  </head>
  <body>                        ← Branch
    <div class="container">     ← Branch
      <h1>Title</h1>            ← Leaf
      <p>Content</p>            ← Leaf
    </div>
  </body>
</html>
```

Element	<tag>content</tag>
Attribute	class="container" id="main"
class	Styling group (multiple allowed)
id	Unique identifier (one per page)
href	Link attribute on <a> tags
src	Source attribute on <img> tags

📌 Note: HTML structure determines HOW you select and extract data.

HTML Tags You'll Encounter When Scraping

```
<!-- Element: <tag>content</tag> -->
<h1>This is a heading</h1>

<!-- Attributes: properties of elements -->
<div class="container" id="main" data-value="123">

<!-- Classes: for styling (can have multiple) -->
<div class="card featured important">

<!-- Links -->
<a href="https://example.com">Click me</a>

<!-- Images -->
<img src="image.jpg" alt="Description">

<!-- Lists -->
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

← class: CSS selector hook
← id: unique per page
← data-*: custom data attributes

← href = 'hyperlink reference'

← alt = alternative text (accessibility)

← ul = unordered list

← li = list item

💡 Pro Tip: Right-click any webpage → 'Inspect' to explore HTML structure live!

Setup — Installing Required Libraries

Install once in your terminal / Jupyter environment:

```
# Web scraping libraries
pip install beautifulsoup4 lxml requests

# Data processing
pip install pandas numpy

# Optional: for dynamic content (Selenium)
pip install selenium
```

```
from bs4 import BeautifulSoup   # Parser
import requests                   # HTTP
import pandas as pd               # Data
```

Package Roles:

`beautifulsoup4`	Parses & navigates HTML/XML
`lxml`	Fast HTML parser (backend for bs4)
`requests`	Sends HTTP GET/POST requests
`pandas`	Structures scraped data
`selenium`	Controls a real browser (JS sites)

✅ **lxml is faster than Python's built-in html.parser — always prefer lxml!**

3.2 CSS Selectors – Finding Elements

CSS selectors are patterns used to target and select HTML elements:

Selector	Example	Meaning
`tag`	`div`	All elements
`.class`	`.post`	Elements with class="post"
`#id`	`#main`	Element with id="main"
`tag.class`	`div.post`	with class='post'
`tag tag`	`div p`	<p> inside (any level)
`tag > tag`	`div > p`	Direct <p> child of
`[attr]`	`[href]`	Elements with href attribute
`[attr^='v']`	`[href^='http']`	Attribute starts with value
`[attr$='v']`	`[src$='.jpg']`	Attribute ends with value
`[attr*='v']`	`[class*='post']`	Attribute contains value
`:first-child`	`li:first-child`	First child element
`:nth-child(n)`	`li:nth-child(2)`	Nth child element

CSS Selectors — Code Examples

```python
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, 'lxml')

# 1. By tag name
titles = soup.select('h2')

# 2. By class (.classname)
posts = soup.select('.post')

# 3. By ID (#id)
post1 = soup.select_one('#post-1')

# 4. Descendant (space = any level deep)
authors = soup.select('.post .author')

# 5. Direct child (> = immediate child only)
container_children = soup.select('.container > article')
```

← Returns list of all matching tags

← Returns list | class selector uses dot prefix

← Returns single element | ID uses # prefix

← Author inside any .post (nested search)

← > restricts to immediate children only

💡 select_one() returns first match or None | select() returns a list (possibly empty)

CSS Selectors — Code Examples

```python
# 6. Multiple classes (no space between them)
featured = soup.select('.post.featured')

# 7. Attribute selector: contains
python_links = soup.select('a[href*="python"]')

# 8. Attribute: starts with
tag_links = soup.select('a[href^="/tag"]')

# 9. Combined selector
post_titles = soup.select('article.post h2.title')

# 10. Nth child
second_post = soup.select_one('article:nth-child(2)')
```

← No space = element must have BOTH classes

← * = contains anywhere in attribute value

← ^ = starts with ($ = ends with)

← Space means descendant at any depth

← nth-child counts siblings of same parent

💡 select_one() returns first match or None | select() returns a list (possibly empty)

3.3 BeautifulSoup – Core Methods

Method	Description
`find_all(tag, attrs)`	Returns list of all matching elements
`select('css_selector')`	CSS selector → list (RECOMMENDED)
`select_one('css_selector')`	CSS selector → first match or None
`.text / .get_text()`	Extract inner text content
`.get('attr', default)`	Get attribute value safely with fallback
`['attr']`	Get attribute value (raises KeyError if missing)
`.parent`	Navigate to parent element
`.find_next_sibling()`	Next sibling in the tree
`.find_previous('tag')`	Previous element of given tag

BeautifulSoup — Extraction Code Examples

```python
# Method 1: find() - returns FIRST match only
first_post = soup.find('article', class_='post')
print(first_post.find('h2').text)

# Method 2: find_all() - returns ALL matches as list
all_posts = soup.find_all('article', class_='post')
for post in all_posts:
    print(post.find('h2').text)

# Method 3: select() - CSS selectors (RECOMMENDED)
posts = soup.select('article.post')

# Extracting data from matched elements
for post in posts:
    title = post.select_one('h2').get_text(strip=True)
    post_id = post.get('id')                    # safe: returns None if missing
    author_id = post.select_one('.author').get('data-id')
    tags = [link.text for link in post.select('.tags a')]
    print(f"ID: {post_id}, Title: {title}")
```

BeautifulSoup — Extraction Code Examples

```python
# Navigate the tree
element = soup.select_one('h2')
parent        = element.parent
next_sibling  = element.find_next_sibling()
prev_div      = element.find_previous('div')

# Handle missing elements SAFELY
price = soup.select_one('.price')
price_text = price.get_text() if price else 'N/A'  # ✅ safe
link = soup.select_one('a')
href = link.get('href', '#') if link else '#'       # ✅ with default
```

⚠️ **Pitfall: Never call .text or .get_text() without checking if element is None first!**

BeautifulSoup — Parsing Strategies

Two ways to feed HTML into BeautifulSoup:

From HTML String (Testing)

```python
html = """
<div class="card">
  <h2>Title</h2>
  <p>Content</p>
</div>
"""
soup = BeautifulSoup(html, 'lxml')
title = soup.select_one('h2').text
print(title)  # → Title
```

From URL (Real Scraping)

```python
import requests
from bs4 import BeautifulSoup

url = 'http://books.toscrape.com'
response = requests.get(url, timeout=10)
response.raise_for_status()  # raises on
4xx/5xx

soup = BeautifulSoup(response.text, 'lxml')
# response.text    → HTML as string
# response.status_code → 200 OK
```

Parser Options:

`lxml`	Fastest & most lenient	✅ **Recommended**
`html.parser`	Built-in Python, no install needed	⚠️ **Slower**
`html5lib`	Most lenient, browser-like	❌ **Very slow**

3.4 Real Example – BookScraper Class

Scenario: Scrape book data from http://books.toscrape.com:

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
from urllib.parse import urljoin

class BookScraper:
    """Scrape book data from books.toscrape.com"""

    def __init__(self):
        self.base_url = 'http://books.toscrape.com'
        self.session = requests.Session()
        # Session: reuses TCP connection → faster than new request each time
        self.session.headers.update({
            'User-Agent': 'Mozilla/5.0 (Educational) BookScraper/1.0'
            # ↑ Identifies your bot to the server (good practice)
        })
```

```
# urljoin(base, relative) safely combines URLs
# urljoin('http://books.toscrape.com', '/catalogue/page-2.html')
# → 'http://books.toscrape.com/catalogue/page-2.html'
```

BookScraper — scrape_page() Method

```python
def scrape_page(self, url):
    """Scrape books from a single page.
    Args:   url  (str)  - URL of the page to scrape
    Returns: tuple - (list of book dicts, next_page_url or None)
    """
    try:
        response = self.session.get(url, timeout=10)
        response.raise_for_status()  # Raises HTTPError for 4xx/5xx codes
        soup = BeautifulSoup(response.text, 'lxml')

        books = []
        for article in soup.select('article.product_pod'):  # each book card
            book = {}

            # Title: stored in <h3><a title="Book Name">…</a></h3>
            title_el = article.select_one('h3 a')
            book['title'] = title_el.get('title')  # use 'title' attr, not .text

            # Price: remove '£' symbol, convert to float
            price_text = article.select_one('.price_color').text
            book['price'] = float(price_text.replace('£', ''))
```

BookScraper — scrape_page() Method

```
        # Availability: check if "In stock" appears in the text
        availability = article.select_one('.availability').text.strip()
        book['in_stock'] = 'In stock' in availability

        # Rating: stored as CSS class e.g. class="star-rating Three"
        rating_el = article.select_one('.star-rating')
        rating_text = rating_el.get('class')[1]   # [0]='star-rating', [1]='Three'
        rating_map = {'One':1,'Two':2,'Three':3,'Four':4,'Five':5}
        book['rating'] = rating_map.get(rating_text, 0)

        # URL: combine relative link with base URL
        book['url'] = urljoin(url, article.select_one('h3 a')['href'])
        books.append(book)

    next_link = soup.select_one('.next a')
    next_url = urljoin(url, next_link['href']) if next_link else None
    return books, next_url

except Exception as e:
    print(f"Error scraping {url}: {e}")
    return [], None
```

BookScraper — Pagination with scrape_all_pages()

```python
def scrape_all_pages(self, max_pages=None):
    """Scrape multiple pages of books.
    Args:
        max_pages (int or None) - Maximum pages to scrape.
                                  None = scrape ALL pages (could be 50+!)
    Returns: list - All book dictionaries combined
    """
    all_books = []
    url = f"{self.base_url}/catalogue/page-1.html"
    page = 1
    while url and (max_pages is None or page <= max_pages):
        print(f"Scraping page {page}...")
        books, next_url = self.scrape_page(url)

        if books:
            all_books.extend(books)      # extend adds all items, not a nested list
            print(f"   Found {len(books)} books")
        url = next_url                   # None when no 'next' link found
        page += 1
        if url:
            time.sleep(1)               # 🔑 polite delay between requests
    print(f"✓ Total books scraped: {len(all_books)}")
    return all_books
```

BookScraper — Fetching Detail Pages & Analysis

```python
def scrape_book_details(self, book_url):
    """Scrape detail page: description, category, UPC."""
    response = self.session.get(book_url, timeout=10)
    soup = BeautifulSoup(response.text, 'lxml')

    desc_el = soup.select_one('#product_description ~ p')
    # ↑ ~ is "adjacent sibling" CSS selector → first <p> after #product_description
    details = {
        'description': desc_el.text if desc_el else 'No description',
        'category': soup.select('.breadcrumb li')[2].text.strip()
        # ↑ breadcrumb[2] is the category (0=Home, 1=Books, 2=Category, 3=Title)
    }
    for row in soup.select('.table.table-striped tr'):
        header = row.select_one('th').text.strip()
        value  = row.select_one('td').text.strip()
        if header == 'UPC':
            details['upc'] = value
        elif header == 'Number of reviews':
            details['num_reviews'] = int(value)
    return details
```

BookScraper — Sample Output & Analysis

Console Output:

```
📖 Scraping page 1...   Found 20 books
📖 Scraping page 2...   Found 20 books
📖 Scraping page 3...   Found 20 books
✓ Total books scraped: 60
```

DataFrame Sample (df.head()):

title	price	rating	in_stock	price_cat
A Light in the Attic	51.77	3	True	Premium
Tipping the Velvet	53.74	1	True	Premium
Soumission	50.10	1	True	Premium

Analysis Report:

```
Total Books: 60
Average Price: £35.72
Range: £10.00 - £59.99

Rating Distribution:
1★: 4   2★: 8   3★: 14   4★: 20   5★: 14

Price Categories:
Mid-range: 28   Budget: 18
Premium: 10     Luxury: 4

📦 In Stock: 58 / Out of Stock: 2
```

3.5 robots.txt – The Rules of Scraping

Always check robots.txt before scraping! It tells you what bots are and are NOT allowed to access.

robots.txt Example (found at http://site.com/robots.txt):

```
User-agent: *          # Applies to ALL bots
Disallow: /admin/      # Don't scrape /admin/
Disallow: /private/    # Don't scrape /private/
Allow: /public/        # /public/ is OK
Crawl-delay: 1         # Wait 1 second between
requests

User-agent: Googlebot # Rules only for Google's bot
Allow: /
```

Directive Meanings:

`User-agent: *`	Applies to every bot
`Disallow: /path/`	Bots must NOT access this path
`Allow: /path/`	Overrides a Disallow for this path
`Crawl-delay: N`	Wait N seconds between requests

3.5 robots.txt — The Rules of Scraping

Python: Check robots.txt Programmatically:

```python
from urllib.robotparser import RobotFileParser
from urllib.parse import urlparse, urljoin

def can_scrape(url, user_agent='*'):
    parsed = urlparse(url)
    robots_url = f"{parsed.scheme}://{parsed.netloc}/robots.txt"
    rp = RobotFileParser()
    rp.set_url(robots_url)  # set the robots.txt URL
    rp.read()               # fetch and parse it
    allowed = rp.can_fetch(user_agent, url)
    print("✅ Allowed" if allowed else "❌ Blocked")
    return allowed

# Usage
can_scrape('http://books.toscrape.com/catalogue/page-1.html')
can_scrape('https://www.google.com/search?q=python')
```

Best Practices & Ethics — What TO DO

✅ DO These Things

✓ **Check robots.txt**	`can_scrape(url) before starting`	Legal & respectful
✓ **Identify your bot**	`Add a User-Agent header with contact info`	Lets site owners reach you
✓ **Add delays**	`time.sleep(random.uniform(1, 3))`	Prevents server overload
✓ **Use sessions**	`requests.Session() reuses connections`	More efficient
✓ **Handle errors**	`try/except with retry logic`	Robust, fault-tolerant code
✓ **Validate data**	`Check types, ranges, duplicates`	Data quality
✓ **Log everything**	`logging module with timestamps`	Debuggable & auditable